

Analyse von Datenbank-Performance, Streuung und Indizierung

David Rinner

28. September 2026

1 Einleitung und Methodik

Ziel dieser Analyse ist die Untersuchung des Verhaltens von SQLite-Datenbanken bei großen Datenmengen ($\geq 500\,000$ Datensätze). Dabei wurden die Streuung von Zufallsdaten, die mithilfe von Python und **Faker** generiert wurden, die Auswirkungen von Indizes auf die Performance sowie der Einfluss eines Daten-Bias untersucht.

2 Streuungsanalyse der Zufallsdaten

Da Namen keine Distanz im herkömmlichen Sinne besitzen, lässt sich ihre Streuung nicht direkt über geometrische Abstände bestimmen. Stattdessen wird die Streuung über die **Häufigkeitsverteilung der Vornamen** definiert.

Jeder Vorname wird dabei als Kategorie betrachtet, deren Häufigkeit über alle Datensätze hinweg gezählt wird.

- **Durchschnittliche Häufigkeit:** $\approx 249,75$ Vorkommen pro Vorname.
- **Varianz der Häufigkeiten:** 267,28 (Standardabweichung $\sigma \approx 16,35$).

Die geringe Varianz zeigt, dass die verwendete Bibliothek **Faker** die Vornamen relativ gleichmäßig über die Datensätze verteilt.

3 Performance und Speicherbedarf mit und ohne Index

Ohne Index muss die Datenbank bei einer Abfrage die gesamte Tabelle durchsuchen und benötigt in der vorliegenden Messung etwa 0,178 s.

Nach der Erstellung eines Indexes sinkt die Abfragezeit auf etwa 0,001 s. Dadurch wird die Abfrage in diesem Beispiel deutlich beschleunigt.

Dieser Geschwindigkeitsgewinn geht jedoch mit einem erhöhten Speicherbedarf einher. Während die Datenbank vor der Erstellung des Indexes 11 755 520 Byte beanspruchte, steigt der Speicherbedarf nach dem Aufbau des B-Tree-Indexes auf 19 578 880 Byte.

Der zusätzliche Speicherbedarf entsteht dadurch, dass neben der eigentlichen Tabelle eine zusätzliche Indexstruktur gespeichert werden muss.

4 Untersuchung der Biased Database

Wird die Verteilung der Vornamen gezielt verändert, sodass 50 % aller Datensätze denselben Vornamen (**Simon**) tragen, verändert sich die Geschwindigkeit der Suche.

- **Seltener Wert (Peter):**
Wird nach einem selten vorkommenden Namen gesucht, kann der Index die passenden Datensätze sehr schnell finden. In der Messung wurde eine Abfragezeit von etwa 0,001 s erreicht.
- **Häufiger Wert (Simon):**
Da **Simon** in 50 % der Datensätze vorkommt, muss die Datenbank sehr viele passende Datensätze finden. Dadurch ist die Suche mit dem Index langsamer als bei einem seltenen Namen. Mit Index benötigt die Abfrage etwa 0,050 s, während ohne Index etwa 0,131 s benötigt werden.

Grund für dieses Verhalten:

Der Unterschied liegt darin, wie oft ein bestimmter Name in der Datenbank vorkommt. Bei einem seltenen Namen wie **Peter** gibt es nur wenige passende Datensätze. Der Index kann diese daher schnell finden.

Bei **Simon** sieht es anders aus: Der Name kommt in der Hälfte aller Datensätze vor. Bei insgesamt 500 000 Datensätzen sind das ungefähr 250 000 Treffer. Der Index muss deshalb deutlich mehr passende Einträge verarbeiten.

Trotzdem ist die Suche mit dem Index in diesem Beispiel noch schneller als die Suche ohne Index. Das zeigt, dass ein Index besonders bei selten vorkommenden Werten hilfreich ist. Bei sehr häufig vorkommenden Werten fällt der Vorteil dagegen kleiner aus.

5 Fazit

Die Untersuchung zeigt, dass die Verwendung eines Indexes bei großen Datenmengen einen erheblichen Einfluss auf die Abfragegeschwindigkeit haben kann. Besonders bei Suchbegriffen mit selten vorkommenden Werten ist der Geschwindigkeitsgewinn deutlich.

Gleichzeitig erhöht ein Index den Speicherbedarf der Datenbank, da zusätzliche Strukturen angelegt und verwaltet werden müssen.

Die Untersuchung der `bias.db` zeigt außerdem, dass die Verteilung der Daten einen wesentlichen Einfluss auf die Effektivität eines Indexes hat. Bei häufig vorkommenden Werten ist der Vorteil eines Indexes geringer als bei seltenen Werten.

Damit wird deutlich, dass bei der Optimierung von Datenbanken nicht nur die Anzahl der Datensätze, sondern auch deren Verteilung und die Selektivität der verwendeten Suchbedingungen berücksichtigt werden müssen.